# Certain low confidence words experiment by GPT-3.5-Turbo

Maryam Naderi

# Idea

In this step, Enno suggested me to ask ChatGPT to only fix certain low-confidence words in the ASR output rather than the whole sentence.

For this experiment, I use GPT-3.5-Turbo 1106 as a language model to correct the ASR errors.

# Here is the prompt of this experiment

```
{       'role': 'system',
        'content': f"""You are a helpful assistant that corrects ASR errors. \
        You will be presented with an ASR transcription and low confidence words in that transcription. \
        the input will be formatted as json with keys text and low_confidence_words,\
        where the text is the ASR transcription and low_confidence_words contains the list of words with low asr confidence. \
        your task is to check if the low confidence words make sense in the transcription and if they do not, replace them with better words. \
        provide your output as a string. \
        Do not change the case, for example, lower case or upper case, in the transcription. \
        Do not output any additional text that is not the corrected transcription. \
        Do not write any explanatory text that is not the corrected transcription.
        """}
```

# Providing example in prompt

{

'role': 'user',

'content': '{"text": "Why not allow your silver tuff to luxuriate in a natural manner?", "low_confidence_words":["tuff"]}'

},

{'role': 'assistant', 'content': "why not allow your silver tufts to luxuriate in a natural manner?"},

{'role': 'user', 'content': json.dumps(asr_transcription)

}

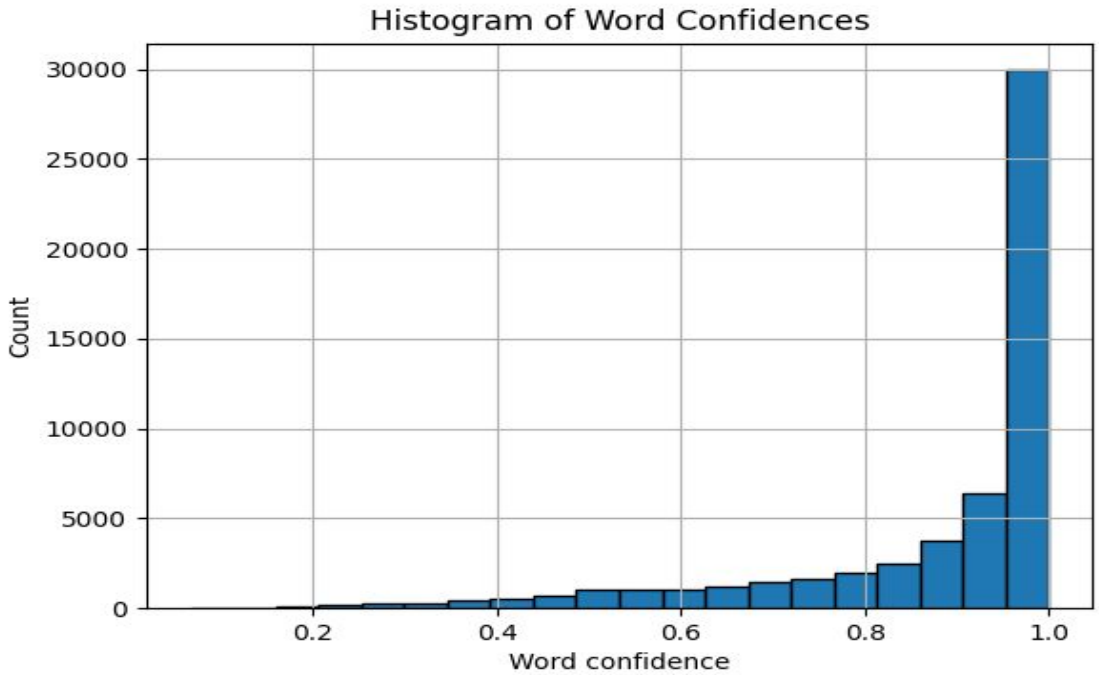
# Code snippet

```
for i,d in enumerate(data):
        asr_transcription = confidence_score_word_level(d["asr_transcription"], confidence = True)
        low_confidence_words = [word["text"] for word in asr_transcription["words"]
                        if word["confidence"] <= THRESH]
        if len(low_confidence_words) > 0:
                asr_transcription = {"text": asr_transcription["text"], "low_confidence_words": low_confidence_words}
                reference_transcription= d["reference_transcription"]
                data1.append({"asr_transcription": asr_transcription, "reference_transcription": reference_transcription})
```

# Here is the one example of output for this experiment


```
{
    "asr_transcription": {
      "text": " But, Dukalion and Piro were very sad. For they knew that they were the only persons who were left alive in all the land.",
      "low_confidence_words": [
        "Dukalion",
        "Piro"
      ]
    },
    "reference_transcription": "BUT DEUCALION AND PYRRHA WERE VERY SAD FOR THEY KNEW THAT THEY WERE THE ONLY PERSONS WHO WERE LEFT ALIVE IN ALL THE LAND",
    "corrected_asr_transcription": "But, Deucalion and Pyrrha were very sad. For they knew that they were the only persons who were left alive in all the land."
  },
```

# Histogram of word confidence



# Results for this experiment by GPT-3.5 Turbo (Thresh =0.5)

```
WER_original:           8.6116%
WER_corrected_chatgpt:  7.5211%
CER_original:           3.5707%
CER_corrected_chatgpt:  3.6174%
SER_original:           61.4502%
SER_corrected_chatgpt:  59.1565%
---
```

# Results for this experiment by GPT-3.5 Turbo (Thresh =0.55)

```
WER_original:           10.2260%
WER_corrected_chatgpt:  9.1994%
CER_original:           4.2068%
CER_corrected_chatgpt:  4.4297%
SER_original:           77.3326%
SER_corrected_chatgpt:  77.4973%
---
```

# Results for this experiment by GPT-3.5 Turbo (Thresh =0.6)

```
WER_original:           9.7043%
WER_corrected_chatgpt:  8.9645%
CER_original:           3.9945%
CER_corrected_chatgpt:  4.3996%
SER_original:           73.6453%
SER_corrected_chatgpt:  75.4680%
---
```

# Results for this experiment by GPT-3.5 Turbo (Thresh =0.65)

```
WER_original:           9.3484%
WER_corrected_chatgpt:  8.7986%
CER_original:           3.8361%
CER_corrected_chatgpt:  4.3280%
SER_original:           70.8711%
SER_corrected_chatgpt:  73.6388%
---
```

# Results for this experiment by GPT-3.5 Turbo (Thresh =0.7)

```
WER_original:           9.0700%
WER_corrected_chatgpt:  8.7170%
CER_original:           3.7202%
CER_corrected_chatgpt:  4.2891%
SER_original:           68.1992%
SER_corrected_chatgpt:  71.6901%
---
```

# Results for this experiment by GPT-3.5 Turbo (Thresh =0.75)

```
WER_original:           8.9594%
WER_corrected_chatgpt:  8.6679%
CER_original:           3.6863%
CER_corrected_chatgpt:  4.3108%
SER_original:           66.7897%
SER_corrected_chatgpt:  70.6847%
---
```

# Results for this experiment by GPT-3.5 Turbo (Thresh =0.8)

```
WER_original:           8.8127%
WER_corrected_chatgpt:  8.5871%
CER_original:           3.6396%
CER_corrected_chatgpt:  4.2491%
SER_original:           64.5972%
SER_corrected_chatgpt:  69.7053%
---
```

# Results for this experiment by GPT-3.5 Turbo (Thresh =0.85)

```
WER_original:           8.7179%
WER_corrected_chatgpt:  8.4257%
CER_original:           3.6077%
CER_corrected_chatgpt:  4.1802%
SER_original:           63.2747%
SER_corrected_chatgpt:  67.4063%
---
```

# Results for this experiment by GPT-3.5 Turbo (Thresh =0.9)

```
WER_original:           8.6424%
WER_corrected_chatgpt:  8.4737%
CER_original:           3.5841%
CER_corrected_chatgpt:  4.2055%
SER_original:           61.9724%
SER_corrected_chatgpt:  65.5585%
---
```

# Results for this experiment by GPT-3.5 Turbo (Thresh =0.95)

```
WER_original:           8.6157%
WER_corrected_chatgpt:  8.5041%
CER_original:           3.5723%
CER_corrected_chatgpt:  4.1458%
SER_original:           61.5185%
SER_corrected_chatgpt:  65.0370%
---
```